خطة تنفيذ نموذج Multispectral Super-Resolution بأعلى أداء ممكن

1. الهدف

الهدف هو بناء نموذج واحد داخل Notebook، لتحسين دقة صورة متعددة الأطياف مكوّنة من أربعة نطاقات، من:

```
4 × 36 × 36 بكسل/متر 2 عند
إلى
4 × 72 × 72 بكسل/متر 1 عند
```

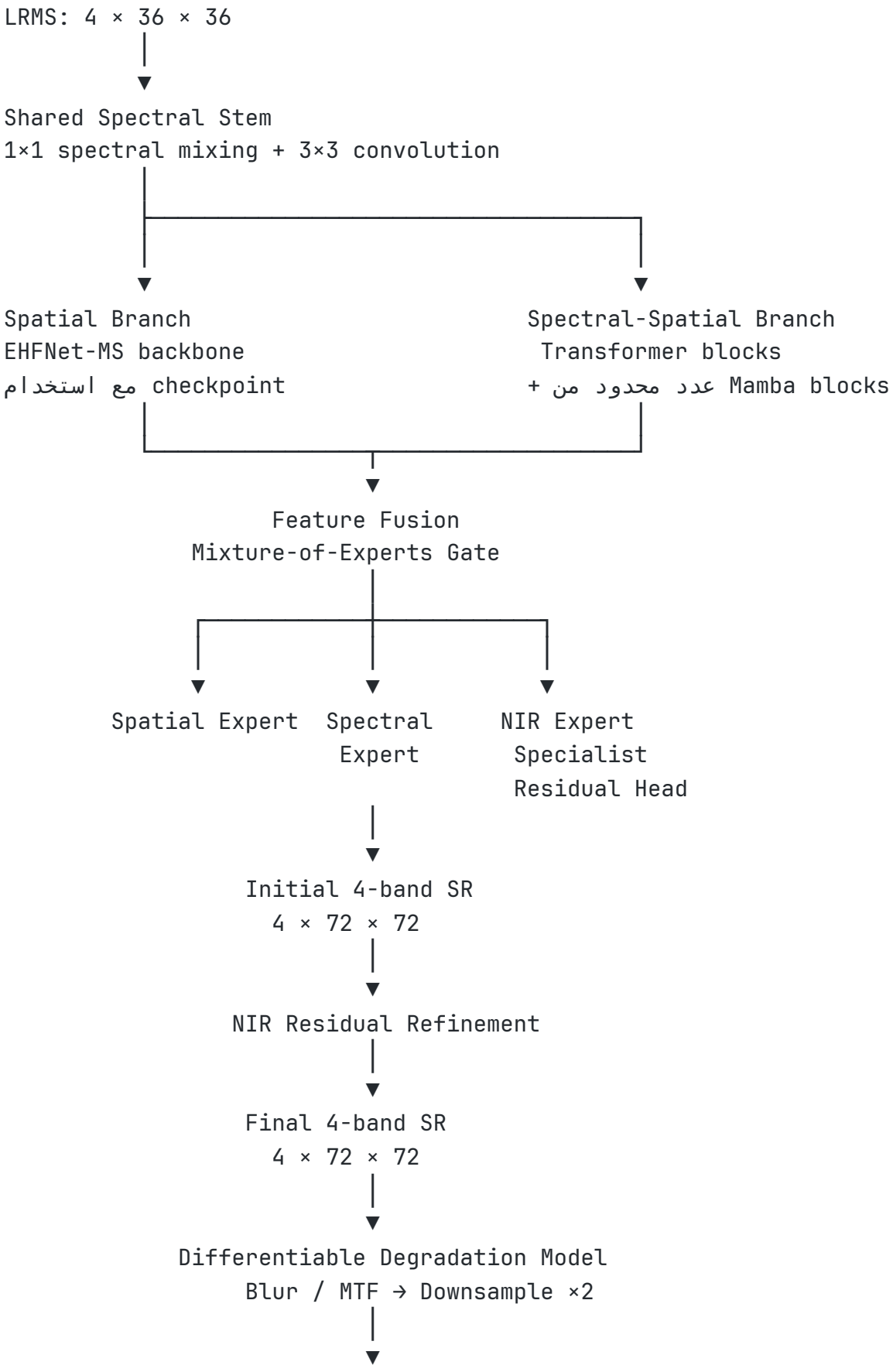
الأولوية ليست للشكل البصري فقط، بل للحصول على أعلى أداء ممكن في الوقت نفسه في:

- الجودة المكانية والتفاصيل الحادة.
- الدقة الطيفية بين النطاقات.
- الخطأ الإشعاعي في نطاق NIR، إذا تم التحقق من أن Band 4 هو NIR.
- تقليل الاختلاق والهلوسة.
- الاتساق مع صورة LR الأصلية بعد تطبيق نموذج الحساس والتصغير.

هذه الخطة مقصودة لتجربة نسخة مركبة هجومية تجمع أكبر عدد عملي من المكونات المهمة، حتى لو لم يكن الهدف الأساسي عزل تأثير كل مكوّن منفردًا.

---

2. المعمارية النهائية المقترحة



```
                Reconstructed LR Image
                          |
                          ▼
Data-Consistency Loss مقابل LR
```

3. المكونات التي سيتم استخدامها

المكوّن	دوره داخل النموذج	القرار
EHFNet-MS pretrained checkpoint	Backbone أساسي لاستخراج التفاصيل المكانية	يستخدم حتمًا
Shared spectral stem	مزج المعلومات بين النطاقات قبل المعالجة الرئيسية	يستخدم
Spatial branch	الحفاظ على الحواف والتفاصيل المكانية	يستخدم
Spectral-spatial Transformer	التقاط العلاقات بعيدة المدى بين البكسلات والنطاقات	يستخدم بعدد محدود من الطبقات
Mamba / State-Space blocks	معالجة العلاقات طويلة المدى بتكلفة أقل من Transformer كبير	يستخدم بعدد محدود
Frequency أو Wavelet branch	تقوية التفاصيل عالية التردد والحواف	يستخدم كفرع residual
NIR specialist	تصحيح NIR بعد الناتج الأولي	يستخدم حتمًا إذا تم التحقق من هوية NIR
Mixture-of-Experts gate	تعلم وزن كل فرع حسب خصائص المنطقة	يستخدم
Uncertainty head	تقدير الثقة في التنبؤ وتوجيه الخسارة	يستخدم
Data-consistency module	إجبار الناتج HR على إعادة إنتاج LR الأصلية	يستخدم حتمًا
Adaptive loss weighting	تعلم أوزان الخسائر أو تعديلها	يستخدم
PPO controller	تعديل أوزان الخسائر كل عدة epochs	تجربة هجومية إضافية

ملاحظة حول Mambaو Transformer

لا يتم وضع عدد كبير من Transformer وMamba معًا. الاستخدام المقترح هو:

```
EHFNet-MS backbone
+ 2 Transformer blocks
+ 1 أو 2 Mamba blocks
```

تعمل هذه الوحدات على feature maps منخفضة الدقة قبل مرحلة upsampling لتقليل استهلاك الذاكرة.

ملاحظة حول Diffusion

لا يتم وضع Diffusion في المسار الأساسي أثناء التدريب الأول؛ لأن ذلك سيزيد الزمن والذاكرة، وقد يضيف تفاصيل غير مدعومة بالقياس. إذا تم استخدامه لاحقًا، يكون كمرحلة refinement منفصلة بعد النموذج الحتمي، وليس داخل المسار الرئيسي.

---

4. فرع NIR المتخصص

النموذج العام ينتج أولًا صورة فائقة الدقة مبدئية:

```
sr_initial = general_sr_head(features)
```

بعد ذلك يأخذ فرع NIR النطاق الرابع والخصائص المشتركة، ويتعلم residual فقط:

```
nir_initial = sr_initial[:, 3:4]
nir_delta = nir_head(fused_features)
nir_final = nir_initial + alpha * torch.tanh(nir_delta)
```

يبدأ `alpha` بقيمة صغيرة:

```
alpha = 0.1 أو 0.2
```

الجيد في بداية التدريب checkpoint ثم يتم رفعه تدريجيًا أثناء التدريب حتى لا يدمّر الـ.

يفضل أن يتنبأ فرع NIR أيضًا بخريطة uncertainty:

```
nir_mean = nir_initial + nir_delta
nir_logvar = uncertainty_head(fused_features)
```

بحيث تحصل المناطق الصعبة على معالجة مختلفة، في حساب خسارة NIR وتستخدم خريطة uncertainty عن المناطق السهلة.

---

5. Mixture-of-Experts Fusion

يتم تعريف ثلاثة خبراء رئيسيين:

```
F_spatial  = خصائص EHFNet-MS
F_spectral = خصائص Transformer وMamba
F_nir      = خصائص فرع NIR
```

تنتج بوابة صغيرة أوزانًا لكل خبير:

```
gate_logits = gate_network(
    torch.cat([F_spatial, F_spectral, F_nir], dim=1)
)

gate = torch.softmax(gate_logits, dim=1)

fused_features = (
    gate[:, 0:1] * F_spatial
    + gate[:, 1:2] * F_spectral
    + gate[:, 2:3] * F_nir
)
```

ويتم استخدام `fused_features` في إنتاج الصورة النهائية.

---

6. الخسارة المركبة

```
L_total = (
    L_charbonnier_all
    + w_nir * L_charbonnier_nir
    + w_sam * L_sam
    + w_bias * L_nir_bias
    + w_inter * L_interband
    + w_edge * L_edge
    + w_freq * L_frequency
    + w_dc * L_data_consistency
    + w_unc * L_uncertainty
    + w_ndvi * L_ndvi
)
```

يتم تشغيل `L_ndvi` فقط بعد التأكد من ترتيب النطاقات ومعرفة Red وNIR بصورة صحيحة.

تعريف العناصر الأساسية

خسارة إعادة البناء

```
L_charbonnier_all = charbonnier(sr_pred - hr_target)
```

خسارة NIR

```
L_charbonnier_nir = charbonnier(
    sr_pred[:, 3:4] - hr_target[:, 3:4]
)
```

خطأ انحياز NIR

```
nir_error = sr_pred[:, 3:4] - hr_target[:, 3:4]
L_nir_bias = torch.abs(nir_error.mean())
```

الاتساق الطيفي

يستخدم SAM على حساب NIR أو خسارة زاوية طيفية، مع خسارة للعلاقات بين النطاقات حتى لا يتحسن بقية النطاقات.

اتساق الحواف والتردد

يتم حساب الفرق بين gradients أو high-frequency maps للناتج والهدف.

الاتساق الفيزيائي

```
lr_reconstructed = degradation_model(sr_pred)
L_data_consistency = charbonnier(
    lr_reconstructed - lr_input
)
```

حيث يحتوي `degradation_model` على:

```
Optical blur أو MTF
ثم downsample 2× بمقياس
```

---

7. أوزان ابتدائية مقترحة

```
w_nir = 1.5
w_sam = 0.10
w_bias = 0.05
w_inter = 0.05
w_edge = 0.05
w_freq = 0.05
w_dc = 0.10
w_unc = 0.02
w_ndvi = 0.0
```

بعد التأكد من استقرار التدريب، يمكن تجربة:

```
w_nir = 2.0 2.5 أو
w_dc = 0.15 0.20 أو
```

لا يتم تشغيل `w_ndvi` إلا بعد التحقق من Red وNIR.

---

8. واحدة Notebook التدريب المرحلي داخل

المرحلة الأولى: تحميل وتجهيز البيانات

1. تحميل LR وHR والتأكد من الترتيب الطيفي.
2. التأكد من normalization نفسه بين LR وHR.
3. التأكد من أن LR وHR يغطيان نفس المساحة الجغرافية.
4. التحقق من أن Band 4 هو NIR قبل استخدام أي خسارة خاصة به.
5. تجهيز random crops وaugmentations مكانية متطابقة لكل النطاقات.

المرحلة الثانية: تهيئة النموذج

1. تحميل checkpoint الخاص بـ EHFNet-MS ×2.
2. إضافة spectral stem.

3. إضافة Transformer/Mamba blocks.
4. إضافة frequency branch.
5. إضافة NIR specialist.
6. إضافة uncertainty head.
7. إضافة Mixture-of-Experts gate.
8. إضافة degradation model.

المرحلة الثالثة: warm start

يتم تجميد معظم الـ backbone فقط وتدريب الفروع الجديدة:

```
تدريب NIR branch
تدريب spectral branch
تدريب frequency branch
تدريب fusion gate
```

يستخدم `alpha` صغيرًا لفرع NIR.

المرحلة الرابعة: fine-tuning

يتم فتح آخر layers من EHFNet-MS، مع learning rate أصغر للـ backbone:

```
LR أعلى :للفروع الجديدة
LR أقل :للـ backbone
```

المرحلة الخامسة: تفعيل القيود

يتم رفع وزن `w_dc` تدريجيًا بدل تفعيله بقوة من أول epoch.

```
w_dc = min(w_dc_max, current_epoch / warmup_epochs * w_dc_max)
```

المرحلة السادسة: adaptive weighting

يتم تفعيل uncertainty weighting أو GradNorm لتعديل أوزان الخسائر.

المرحلة السابعة: RL Controller

يتم تشغيل PPO controller خارجيًا كل 5 أو 10 epochs. وظيفته تعديل فقط، ولا يغير أوزان الشبكة نفسها.
أوزان الخسائر ضمن حدود صغيرة.

---

9. إدخال RL Controller

الحالة State

```
validation NIR RMSE
validation NIR MAE
NIR Bias
SAM
spectral correlation
PSNR
SSIM
edge error
frequency error
data-consistency error
current loss weights
```

الفعل Action

يعدل الوكيل أوزان الخسائر بنسبة محدودة:

```
lambda_new = clamp(
    lambda_old * torch.exp(action),
    lambda_min,
    lambda_max
)
```

البيئة Environment

البيئة هي دورة تدريب قصيرة للنموذج لمدة 5–10 epochs، ثم حساب metrics على validation set.

المكافأة Reward

```
Reward =
    تحسن composite validation score
    - عقوبة إذا ساء NIR Bias
    - عقوبة إذا زاد data-consistency error
    - عقوبة إذا انهارت الجودة المكانية
```

القيود

- لا يغير RL المعمارية.
- لا يغير parameters الشبكة مباشرة.
- لا يغير الأوزان كل batch.
- لا يغير أكثر من نسبة صغيرة في كل خطوة.
- لا يعمل قبل استقرار النموذج الأساسي.

يسبب عدم استقرار أو يأخذ وقتًا مبالغًا فيه، يتم الاحتفاظ ببقية النموذج وإيقاف الـ PPO إذا كان استخدام Controller فقط.

---

10. اختبارات الجودة والـ Metrics

يتم حفظ metrics التالية لكل validation epoch:

الفئة	المقاييس
Spatial	PSNR، SSIM، edge error، high-frequency error
Pixel	MAE، RMSE، Bias لكل band
NIR	NIR MAE، NIR RMSE، NIR Bias، NIR PSNR، NIR SSIM
Spectral	SAM، spectral correlation، spectral distortion
Physical	degradation consistency error، LR reconstruction error
Vegetation	NDVI MAE وRMSE وBias فقط بعد التحقق من NIR وRed

يتم استخدام score مركب لاختيار أفضل نموذج:

```
OverallScore =
    0.35 × normalized NIR RMSE
  + 0.20 × normalized NIR Bias
  + 0.20 × normalized SAM
  + 0.15 × normalized Data Consistency Error
  + 0.10 × normalized Spatial Error
```

كلما انخفض `OverallScore` كان النموذج أفضل.

---

11. إعدادات عملية للـ RTX A6000

- استخدام mixed precision.
- استخدام gradient accumulation إذا لم يتسع الـ batch.
- وضع Transformer وMamba على feature maps منخفضة الدقة.
- استخدام gradient clipping، مثل `max_norm=1.0`.
- استخدام EMA للـ model weights إذا كان مستقرًا.
- استخدام checkpointing للطبقات الثقيلة عند الحاجة.
- عدم تطبيق augmentations تغيّر العلاقات الطيفية بين النطاقات.
- استخدام overlap أثناء tiled inference لتقليل آثار حدود البلاطات.
- حفظ أفضل checkpoint بشكل مستقل أثناء التدريب.

---

12. Pseudocode للنموذج النهائي

```
# Input: lr_image [B, 4, 36, 36]
# Target: hr_image [B, 4, 72, 72]

x = spectral_stem(lr_image)

f_spatial = ehfnet_backbone(x)
f_spectral = spectral_transformer(x)
f_spectral = mamba_blocks(f_spectral)
f_frequency = frequency_branch(x)

f_spatial = f_spatial + f_frequency

f_nir = nir_feature_encoder(x, f_spatial)

# Match feature dimensions if needed
f_spatial, f_spectral, f_nir = align_features(
    f_spatial, f_spectral, f_nir
)

gate = moe_gate(torch.cat([
    f_spatial, f_spectral, f_nir
], dim=1))

fused = (
    gate[:, 0:1] * f_spatial
    + gate[:, 1:2] * f_spectral
    + gate[:, 2:3] * f_nir
)

sr_initial = general_sr_head(fused)

nir_initial = sr_initial[:, 3:4]
nir_delta = nir_residual_head(fused)
nir_final = nir_initial + alpha * torch.tanh(nir_delta)

sr_final = sr_initial.clone()
sr_final[:, 3:4] = nir_final

uncertainty = uncertainty_head(fused)

lr_reconstructed = degradation_model(sr_final)
```

```
loss = composite_loss(
    sr_final=sr_final,
    hr_target=hr_image,
    lr_input=lr_image,
    lr_reconstructed=lr_reconstructed,
    uncertainty=uncertainty,
    loss_weights=loss_weights
)

loss.backward()
clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

---

13. القرار النهائي

النموذج الذي سيتم تجربته هو:

> **EHFNet-MS pretrained backbone + spectral stem + Transformer + عدد محدود من Mamba blocks + frequency branch + NIR residual specialist + uncertainty head + Mixture-of-Experts fusion + differentiable physical consistency + adaptive loss weighting + PPO loss controller.**

يتم استخدام RL، لكن كـ **Controller** وليس كبديل عن الشبكة أو كـ **خارجي لأوزان الخسائر**، Multi-Agent RL.

لا يتم وضع Diffusion وMulti-Agent RL في المسار الأساسي؛ لأنهما سيزيدان التعقيد والتكلفة أكثر من الفائدة المتوقعة في Notebook واحدة. أما باقي المكونات فتُجمع داخل نموذج واحد مع warm-start وstaged training للحفاظ على استقرار التدريب.

14. الخلاصة التنفيذية

```
ابدأ بالـ checkpoint الحالي
        ↓
أضف كل الفروع الجديدة
        ↓
درّب الفروع مع backbone شبه مجمّد
        ↓
افتح الـ backbone تدريجيًا
        ↓
فعّل physical consistency
        ↓
فعّل adaptive loss weighting
        ↓
شغّل PPO controller كل عدة epochs
        ↓
fine-tuning نهائي
        ↓
اختيار best_overall.pt
```

هذا هو التصميم العدواني المقترح لتعظيم النتيجة النهائية مع الحفاظ على إمكانية تشغيله داخل Notebook واحدة وعلى RTX A6000 48 بذاكرة GB.